

Day 6

Advanced LangGraph - Human Review, Error Recovery and Multi-Agent Flow

Insurance Claims Settlement Workflow

Program Objective

- Build a multi-agent claim settlement workflow with human review checkpoints.
- Apply LangChain memory, tools and LLM calls in insurance automation scenarios.
- Use LangGraph for approval, rejection, escalation, rework and recovery flows.
- Design practical workflows that can later integrate with UiPath and enterprise systems.

Audience	Prerequisites	Final Deliverable
UiPath developers, RPA engineers, automation leads, AI automation teams	Python basics, LLM API access, Day 5 LangGraph foundation	Multi-Agent Claim Settlement Workflow with human review

1. Day 6 Learning Outcomes

- Explain LangChain versus LangGraph using insurance automation examples.
- Create a LangChain insurance assistant with memory, tools and LLM model usage.
- Use LangGraph state, checkpoints, branching and human review concepts.
- Create approval, rejection, escalation and rework routes for claims.
- Add retry and error recovery patterns for LLM/API failures.
- Build medical, motor and travel claim routing flows.
- Design a complete multi-agent claim settlement workflow with human review.

2. Day 6 Session Plan

Time	Topic	Hands-on Focus
30 min	LangChain Agentic AI use case	Memory, tools and LLM example for insurance
25 min	LangChain vs LangGraph	When to use chains and when to use graphs
35 min	Human-in-the-loop review patterns	High-value claim checkpoint
35 min	Approval, rejection, escalation and rework routing	Claim settlement decision branches
30 min	Memory and state updates	Store claim history and reviewer notes
30 min	Retry and error recovery	API retry and fallback review route
45 min	Multi-agent collaboration	Medical, motor and travel claim routing
30 min	Final assignment	Build complete settlement workflow

Trainer Note

- Keep the session practical. Explain one concept, immediately run one insurance example.
- Use human review as a governance concept, not only a technical concept.
- Humour: “An agent can recommend. A manager signs. Insurance likes signatures.”

3. Required Setup

Participants are assumed to have working API access. The following environment variables are already configured:

```

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = os.getenv("OPENROUTER_BASE_URL",
"https://openrouter.ai/api/v1")
OPENROUTER_MODEL = os.getenv("OPENROUTER_MODEL",
"openai/gpt-4.1-mini")

```

Required Python packages:

- openai
- python-dotenv
- langchain
- langchain-openai
- langgraph
- pydantic
- typing_extensions
- pandas

```

# Install only if required
pip install openai python-dotenv langchain langchain-openai
langgraph pydantic pandas

```

4. Concept: LangChain Agentic AI Use Case for Insurance Domain

LangChain is useful for building LLM-powered assistants that use prompts, tools, memory and structured outputs. In insurance automation, LangChain can support customer service, claim status explanations, policy FAQ, document checklist generation and underwriting support.

Component	Meaning	Insurance Example
LLM Model	The language model used for reasoning or generation	Explain claim status in customer-friendly language
Prompt	Instruction given to the model	Summarize claim note and list next action
Tool	A function the assistant can use	Policy lookup, claim status lookup, premium check
Memory	Conversation or workflow history	Previous customer questions and claim follow-ups
Chain	Reusable LLM workflow	Policy FAQ chain or claim summary chain

5. Hands-on Lab 1: LangChain with Memory, Tools and LLM Model

Business scenario: A customer asks repeated questions about claim CLM9002. The assistant should remember the conversation, use a claim status tool and respond professionally.

Step 1: Configure the OpenAI-compatible LLM client.

```
import os, json
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = os.getenv("OPENROUTER_BASE_URL",
    "https://openrouter.ai/api/v1")
OPENROUTER_MODEL = os.getenv("OPENROUTER_MODEL",
    "openai/gpt-4.1-mini")

client = OpenAI(base_url=OPENROUTER_BASE_URL,
    api_key=OPENROUTER_API_KEY)

def call_llm(prompt, system_message="You are an insurance
automation assistant.", temperature=0.2):
    response = client.chat.completions.create(
        model=OPENROUTER_MODEL,
        messages=[
            {"role": "system", "content": system_message},
            {"role": "user", "content": prompt}
        ],
        temperature=temperature
    )
    return response.choices[0].message.content
```

Step 2: Create mock claim data and a claim status tool.

```
claim_status_database = {
    "CLM9001": {"status": "Document Verification",
    "pending": ["Claim Form"], "amount": 85000},
    "CLM9002": {"status": "Pending Documents", "pending":
    ["Police Report", "Driving License"], "amount": 240000}
```

```

}

def claim_status_tool(claim_id):
    return claim_status_database.get(claim_id, {"error":
"Claim not found"})

```

Step 3: Add lightweight memory using a list. This is simple and beginner-friendly.

```

conversation_memory = []

def insurance_memory_assistant(user_question, claim_id):
    tool_result = claim_status_tool(claim_id)
    conversation_memory.append({"role": "user", "content":
user_question})

    prompt = f"""
You are an insurance claim support assistant.
Use the claim status tool result and conversation history.
Do not promise claim approval.

Conversation History:
{json.dumps(conversation_memory, indent=2)}

Claim Tool Result:
{json.dumps(tool_result, indent=2)}

User Question:
{user_question}
"""

    answer = call_llm(prompt)
    conversation_memory.append({"role": "assistant",
"content": answer})
    return answer

print(insurance_memory_assistant("What is the status of my
claim?", "CLM9002"))
print(insurance_memory_assistant("What documents are
pending?", "CLM9002"))

```

Mini Assignment 1

- Add a policy status tool and allow the assistant to answer both claim and policy questions.
- Store the last three customer questions only.
- Add a rule: Never mention internal risk labels to customers.

6. Concept: LangChain vs LangGraph

Area	LangChain	LangGraph
Best For	Reusable prompt workflows, tools, memory and simple assistants	Stateful multi-step and multi-agent workflows
Control Flow	Mostly linear or tool-driven	Graph-based with branches and loops
State	Can use memory and variables	State is central and explicit
Human Review	Possible but usually custom	Designed for checkpoints and controlled routing
Insurance Example	Policy FAQ assistant	Claim settlement workflow with approval/rework

Simple decision rule: Use LangChain when you need an assistant. Use LangGraph when you need a controlled workflow with decisions, review, loops and state.

7. Concept: LangGraph Human-in-the-Loop Review Patterns

Human-in-the-loop means the workflow intentionally stops or routes to a human reviewer when automation should not make the final decision.

Pattern	When to Use	Insurance Example
Approval checkpoint	Decision requires authorization	Claim amount above INR 5,00,000
Rework loop	More information is required	Missing discharge summary or police report
Escalation	Risk or exception exists	High risk motor claim
Manual override	Human decision replaces automation recommendation	Senior claims manager approves exception
Audit note capture	Decision must be explainable	Reviewer records approval reason

Practical Explanation

- Human review is not a failure of automation. It is a governance feature.
- In insurance, automation should prepare the case. Humans approve sensitive decisions.
- Humour: “The agent can say ready. The claim manager says really?”

8. Hands-on Lab 2: Build High-Value Claim Human Review Checkpoint

Business rule: Any claim above INR 5,00,000 must go to human review before settlement.

```
from typing import TypedDict, Dict, Any, List
from langgraph.graph import StateGraph, START, END

class SettlementState(TypedDict, total=False):
    claim: Dict[str, Any]
    settlement_recommendation: str
    requires_human_review: bool
    reviewer_decision: str
    reviewer_notes: str
    final_status: str
    logs: List[str]

def claim_assessment_agent(state: SettlementState) -> SettlementState:
    claim = state["claim"]
    amount = claim.get("claim_amount", 0)
    requires_review = amount > 500000
    recommendation = "Human Review Required" if
requires_review else "Eligible for Auto Settlement Review"
    logs = state.get("logs", []) + ["Claim Assessment Agent
completed."]
    return {**state, "requires_human_review":
requires_review, "settlement_recommendation":
recommendation, "logs": logs}

def human_review_checkpoint(state: SettlementState) ->
```

```
SettlementState:
```

```
    # In classroom practice, reviewer decision is
    simulated.
```

```
    # In production, this can come from UiPath Action
    Center, web UI, ServiceNow, or CRM.
```

```
    reviewer_decision = "Approved for Settlement"
```

```
    reviewer_notes = "High-value claim reviewed. Documents
    and policy status verified."
```

```
    logs = state.get("logs", []) + ["Human Review
    Checkpoint completed."]
```

```
    return {**state, "reviewer_decision":
    reviewer_decision, "reviewer_notes": reviewer_notes,
    "logs": logs}
```

```
def auto_settlement_node(state: SettlementState) ->
    SettlementState:
```

```
    logs = state.get("logs", []) + ["Auto Settlement Node
    completed."]
```

```
    return {**state, "final_status": "Proceed with
    automated settlement processing", "logs": logs}
```

```
def final_approval_node(state: SettlementState) ->
    SettlementState:
```

```
    logs = state.get("logs", []) + ["Final Approval Node
    completed."]
```

```
    return {**state, "final_status":
    state.get("reviewer_decision"), "logs": logs}
```

```
def review_router(state: SettlementState) -> str:
```

```
    return "human_review" if
    state.get("requires_human_review") else "auto_settlement"
```

```
workflow = StateGraph(SettlementState)
```

```
workflow.add_node("claim_assessment",
    claim_assessment_agent)
```

```
workflow.add_node("human_review", human_review_checkpoint)
```

```
workflow.add_node("auto_settlement", auto_settlement_node)
```

```
workflow.add_node("final_approval", final_approval_node)
```

```

workflow.add_edge(START, "claim_assessment")
workflow.add_conditional_edges(
    "claim_assessment",
    review_router,
    {"human_review": "human_review", "auto_settlement":
"auto_settlement"}
)
workflow.add_edge("human_review", "final_approval")
workflow.add_edge("auto_settlement", END)
workflow.add_edge("final_approval", END)

settlement_app = workflow.compile()

high_value_claim = {"claim_id": "CLM9101", "claim_type":
"Medical", "claim_amount": 750000}
result = settlement_app.invoke({"claim": high_value_claim,
"logs": []})
print(result["final_status"])
print(result["logs"])

```

Mini Assignment 2

- Change reviewer_decision to Rejected and observe the final status.
- Add a reviewer_name field to the state.
- Add condition: Motor claims above INR 2,00,000 must also go to human review.

9. Concept: Approval, Rejection, Escalation and Rework Routing

Route	Meaning	Insurance Example
Approval	Claim can move forward	Valid medical claim with complete documents
Rejection	Claim cannot proceed based on rules	Inactive policy or non-covered treatment
Escalation	Senior review is required	High risk or high-value claim
Rework	Claim returns for correction or more data	Missing document or unclear invoice
Manual Review	Default safe route	Ambiguous cases

10. Hands-on Lab 3: Approval, Rejection, Escalation and Rework Branches

Business scenario: After review, claim moves to one of four routes: approved, rejected, escalated or rework.

```
def decision_agent(state: SettlementState) -> SettlementState:
    claim = state["claim"]
    amount = claim.get("claim_amount", 0)
    policy_status = claim.get("policy_status", "Active")
    missing_documents = claim.get("missing_documents", [])
    risk_level = claim.get("risk_level", "Low")

    if policy_status != "Active":
        decision = "reject"
    elif missing_documents:
        decision = "rework"
    elif risk_level == "High" or amount > 500000:
        decision = "escalate"
    else:
        decision = "approve"

    logs = state.get("logs", []) + [f"Decision Agent
selected route: {decision}"]
    return {**state, "reviewer_decision": decision, "logs":
logs}

def approval_node(state):
    return {**state, "final_status": "Approved for
settlement"}

def rejection_node(state):
    return {**state, "final_status": "Rejected due to
policy or eligibility issue"}

def escalation_node(state):
    return {**state, "final_status": "Escalated to senior
```

```
claims_manager"]}
```

```
def rework_node(state):
    return {**state, "final_status": "Returned for rework:
collect missing documents"}
```

```
def decision_router(state):
    return state.get("reviewer_decision", "rework")
```

Mini Assignment 3

- Add a `rework_count` field and stop rework after two attempts.
- Route third rework attempt to manual review.
- Generate a customer email for rework cases using `call_llm()`.

11. Concept: Memory and State Updates

Memory stores conversation or previous interactions. State stores current workflow data. In LangGraph, state is updated at every node, making it suitable for audit and decision tracking.

Type	Purpose	Example
Memory	Keeps interaction history	Customer asked twice about same missing document
State	Keeps workflow facts and decisions	risk_level = High, route = escalation
Audit Log	Keeps execution trail	Policy agent completed, reviewer approved
Reviewer Notes	Keeps human decision reason	Approved after document verification

```
def update_memory_node(state):
    memory = state.get("memory", [])
    memory.append({
        "event": "claim_review_update",
        "claim_id": state["claim"].get("claim_id"),
        "status": state.get("final_status"),
        "notes": state.get("reviewer_notes", "")
    })
```

```

Day 6 - Advanced LangGraph: Human Review, Error Recovery and Multi-Agent Flow
logs = state.get("logs", []) + ["Memory updated."]
return {**state, "memory": memory, "logs": logs}

```

Mini Assignment 4

- Store every routing decision in memory.
- Add timestamp to every memory entry.
- Create a function to print complete claim audit trail.

12. Concept: Retry and Error Recovery Strategies

LLM and API calls can fail. Enterprise workflows should recover gracefully instead of crashing.

Failure Type	Recovery Strategy	Insurance Workflow Action
Temporary API failure	Retry with delay	Retry claim summary generation
Invalid JSON response	Ask model to regenerate JSON	Recreate risk assessment output
Missing tool response	Route to manual review	Do not continue settlement
Repeated failure	Escalate	Send case to technical support or operations
Timeout	Fallback response	Mark AI service unavailable

13. Hands-on Lab 4: Add Retry Logic for API Failures

```

import time

def call_llm_with_retry(prompt, max_retries=3,
                        delay_seconds=2):
    last_error = None
    for attempt in range(1, max_retries + 1):
        try:
            return call_llm(prompt)
        except Exception as error:
            last_error = error
            print(f"Attempt {attempt} failed: {error}")
            if attempt < max_retries:
                time.sleep(delay_seconds * attempt)

```

```

return f"AI service failed after retries. Route to
manual review. Error: {last_error}"

def safe_claim_summary_agent(state):
    claim = state["claim"]
    prompt = f"Summarize this insurance claim for
operations team:
{json.dumps(claim, indent=2)}"
    summary = call_llm_with_retry(prompt)
    logs = state.get("logs", []) + ["Safe Claim Summary
Agent completed."]
    return {**state, "claim_summary": summary, "logs":
logs}

```

Mini Assignment 5

- Add a `retry_count` value to state.
- If `retry_count` exceeds 3, route to Technical Exception Queue.
- Log the error message in workflow logs.

14. Concept: Multi-Agent Collaboration for Complex Insurance Workflows

Complex claim settlement requires multiple specialized agents. Each agent should have a clear responsibility and should not cross decision boundaries.

Agent	Responsibility	Boundary
Claim Intake Agent	Summarize claim and identify claim type	Should not approve settlement
Document Agent	Check missing documents	Should not evaluate fraud
Policy Agent	Validate policy, premium and coverage	Should not override policy data
Medical/Motor/Travel Agent	Apply domain-specific rules	Should not finalize payment
Risk Agent	Identify risk indicators	Should not accuse customer
Human Review Agent	Capture reviewer decision	Should log reasons
Settlement Agent	Prepare final settlement status	Should follow approved decision

15. Hands-on Lab 5: Medical / Motor / Travel Claim Routing

Business scenario: Claims are first classified by type and then sent to specialized claim agents.

```
def claim_type_router(state):
    claim_type = state["claim"].get("claim_type",
    "Unknown")
    if claim_type == "Medical":
        return "medical"
    if claim_type == "Motor":
        return "motor"
    if claim_type == "Travel":
        return "travel"
    return "manual"

def medical_claim_agent(state):
    claim = state["claim"]
    required = ["Hospital Bill", "Discharge Summary",
    "Doctor Prescription", "Claim Form"]
    missing = [d for d in required if d not in
    claim.get("documents", [])]
    route = "rework" if missing else "medical_review"
    logs = state.get("logs", []) + ["Medical Claim Agent
    completed."]
    return {**state, "missing_documents": missing,
    "domain_route": route, "logs": logs}

def motor_claim_agent(state):
    claim = state["claim"]
    required = ["Repair Estimate", "Vehicle Photos",
    "Police Report", "Driving License"]
    missing = [d for d in required if d not in
    claim.get("documents", [])]
    route = "risk_review" if "Police Report" in missing
    else "motor_review"
    logs = state.get("logs", []) + ["Motor Claim Agent
    completed."]
    return {**state, "missing_documents": missing,
```

```

"domain_route": route, "logs": logs}

def travel_claim_agent(state):
    claim = state["claim"]
    required = ["Boarding Pass", "Airline Certificate",
"Baggage Delay Report"]
    missing = [d for d in required if d not in
claim.get("documents", [])]
    route = "rework" if missing else "travel_review"
    logs = state.get("logs", []) + ["Travel Claim Agent
completed."]
    return {**state, "missing_documents": missing,
"domain_route": route, "logs": logs}

```

Mini Assignment 6

- Add a Property Claim Agent with property-specific documents.
- Create separate route for catastrophic damage claims.
- Add business rule: claims above INR 10,00,000 always need senior review.

16. Complete Example: Multi-Agent Claim Settlement Workflow

This workflow combines intake, domain routing, specialized claim agents, risk review, human review, approval/rejection and final settlement status.

```

class AdvancedClaimState(TypedDict, total=False):
    claim: Dict[str, Any]
    claim_summary: str
    missing_documents: List[str]
    domain_route: str
    risk_level: str
    human_review_required: bool
    reviewer_decision: str
    final_status: str
    logs: List[str]

def intake_agent(state):
    claim = state["claim"]
    summary = f"{claim.get('claim_type')} claim
{claim.get('claim_id')} for INR {claim.get('claim_amount')}"

```

```
received."
```

```
    return {**state, "claim_summary": summary, "logs":
state.get("logs", []) + ["Intake Agent completed."]}
```

```
def risk_agent(state):
```

```
    amount = state["claim"].get("claim_amount", 0)
```

```
    missing = state.get("missing_documents", [])
```

```
    if amount > 500000 or len(missing) > 2:
```

```
        risk = "High"
```

```
    elif missing:
```

```
        risk = "Medium"
```

```
    else:
```

```
        risk = "Low"
```

```
    return {**state, "risk_level": risk, "logs":
```

```
state.get("logs", []) + ["Risk Agent completed."]}
```

```
def human_review_decision_agent(state):
```

```
    required = state.get("risk_level") == "High" or
```

```
state["claim"].get("claim_amount", 0) > 500000
```

```
    decision = "Approved" if required else "Not Required"
```

```
    return {**state, "human_review_required": required,
```

```
"reviewer_decision": decision, "logs": state.get("logs",
[])) + ["Human Review Decision Agent completed."]}
```

```
def settlement_node(state):
```

```
    if state.get("missing_documents"):
```

```
        status = "Rework Required - Missing Documents"
```

```
    elif state.get("reviewer_decision") == "Approved" or
```

```
not state.get("human_review_required"):
```

```
        status = "Approved for Settlement Processing"
```

```
    else:
```

```
        status = "Manual Review Required"
```

```
    return {**state, "final_status": status, "logs":
```

```
state.get("logs", []) + ["Settlement Node completed."]}
```

```
advanced_workflow = StateGraph(AdvancedClaimState)
```

```
advanced_workflow.add_node("intake", intake_agent)
```

```
advanced_workflow.add_node("medical", medical_claim_agent)
```

```

advanced_workflow.add_node("motor", motor_claim_agent)
advanced_workflow.add_node("travel", travel_claim_agent)
advanced_workflow.add_node("risk", risk_agent)
advanced_workflow.add_node("human_review",
human_review_decision_agent)
advanced_workflow.add_node("settlement", settlement_node)

advanced_workflow.add_edge(START, "intake")
advanced_workflow.add_conditional_edges(
    "intake",
    claim_type_router,
    {"medical": "medical", "motor": "motor", "travel":
"travel", "manual": "settlement"}
)
advanced_workflow.add_edge("medical", "risk")
advanced_workflow.add_edge("motor", "risk")
advanced_workflow.add_edge("travel", "risk")
advanced_workflow.add_edge("risk", "human_review")
advanced_workflow.add_edge("human_review", "settlement")
advanced_workflow.add_edge("settlement", END)

advanced_claim_app = advanced_workflow.compile()

sample_medical_claim = {
    "claim_id": "CLM9201", "claim_type": "Medical",
    "claim_amount": 650000,
    "documents": ["Hospital Bill", "Discharge Summary",
    "Doctor Prescription", "Claim Form"]
}
result = advanced_claim_app.invoke({"claim":
sample_medical_claim, "logs": []})
print(result["final_status"])
print(result["logs"])

```

17. Additional Example 1: Missing Document Escalation Flow

This example simplifies the concept of rework routing. If documents are missing, the claim does not proceed to settlement. It goes to document collection.

```
def missing_document_escalation_node(state):
    missing = state.get("missing_documents", [])
    message = f"Escalate to document collection team.
Missing: {' '.join(missing)}"
    return {**state, "final_status": message}

# Practice: Connect this node before settlement if
missing_documents is not empty.
```

Practice Task

- Create a route named document_escalation.
- Send medical claims with missing discharge summary to this route.
- Generate a customer email asking for missing documents.

18. Additional Example 2: Approval and Rejection Branches

```
def settlement_decision_router(state):
    if state.get("missing_documents"):
        return "rework"
    if state.get("risk_level") == "High":
        return "human_review"
    if state.get("claim", {}).get("policy_status") ==
"Inactive":
        return "reject"
    return "approve"
```

Practice Task

- Add four branches: approve, reject, rework, human_review.
- Create final messages for each branch.
- Add reviewer_notes to human_review branch.

19. Additional Example 3: LLM Model in LangGraph Node

Use the LLM inside a LangGraph node when text reasoning is needed, such as settlement explanation, customer communication or risk summary.

```
def llm_settlement_explanation_agent(state):
    prompt = f"""
You are an insurance settlement assistant.
```

Create a short internal settlement explanation.
Do not promise customer approval.

State:

```
{json.dumps(state, indent=2)}  
"""  
    explanation = call_llm_with_retry(prompt)  
    return {**state, "settlement_explanation": explanation}
```

Practice Task

- Create an LLM customer email node.
- Create an LLM internal audit summary node.
- Use neutral language for risk and rejection cases.

20. Additional Example 4: Error Recovery Route

If an LLM node fails or returns invalid output, route the claim to Technical Exception Queue or Manual Review Queue.

```
def safe_json_llm_agent(state):  
    prompt = "Return JSON with risk_level and reason for  
this claim."  
    response = call_llm_with_retry(prompt)  
    try:  
        parsed = json.loads(response)  
        return {**state, "risk_level":  
parsed.get("risk_level", "Medium")}  
    except Exception:  
        return {**state, "risk_level": "Unknown",  
"technical_exception": True}  
  
def error_router(state):  
    return "technical_exception" if  
state.get("technical_exception") else "continue"
```

Practice Task

- Create a Technical Exception Node.
- Store the raw LLM response for debugging.
- Do not continue settlement when output is invalid.

21. Assignment: Multi-Agent Claim Settlement Workflow with Human Review

Build a complete workflow that accepts medical, motor and travel claims and routes them through multiple agents with human review and recovery logic.

Required Component	Expected Output
Claim Intake Agent	Claim summary and basic claim facts
Claim Type Router	Medical, motor, travel or manual route
Domain-Specific Agent	Missing documents and claim-specific review flags
Risk Agent	Low, Medium or High risk level
Human Review Checkpoint	Reviewer decision and notes for high-value/high-risk claims
Retry/Error Recovery	Fallback route for API failures or invalid output
Settlement Node	Approved, rejected, rework or escalated final status
Audit Log	Step-by-step workflow execution trail

22. Final Assignment Checklist

- Create AdvancedClaimState schema.
- Create intake, medical, motor and travel agents.
- Add missing document escalation route.
- Add risk review agent.
- Add human review checkpoint for high-value claims.
- Add approval, rejection, escalation and rework branches.
- Add retry logic for LLM/API failure.
- Use `call_llm()` in at least one LangGraph node.
- Test medical, motor and travel claims.
- Print final status and logs for each test case.
- Prepare short explanation of agent responsibilities.

23. Discussion Questions

1. Where should human review be mandatory in insurance workflows?
2. What is the difference between memory and state?
3. When should a workflow retry and when should it escalate?
4. Which agent should be allowed to make settlement recommendations?
5. How can UiPath Action Center be used as a human review interface?
6. What should be logged for audit and compliance?

7. How can this workflow connect with CRM, policy and claims systems?

24. Day 6 Summary

Key Takeaways

- LangChain is useful for assistants, tools and memory-based interactions.
- LangGraph is useful for stateful, governed and branch-based workflows.
- Human review is essential for high-value, high-risk and ambiguous claims.
- Retry and error recovery are mandatory for enterprise-grade AI automation.
- Multi-agent workflows should use clear responsibilities and boundaries.
- The final workflow should support approval, rejection, escalation and rework.

25. Suggested Next Step

Participants should now implement the Day 6 workflow in Jupyter Notebook and then expose the final workflow through FastAPI so UiPath can call it using HTTP Request activity. This will prepare them for enterprise integration and capstone development.